

Adestramento de modelos de IA

Laura Cabaleiro Pintos & Pablo Chantada Saborido

Marcelo Ferreiro Sánchez & José Romero Conde

Curso 2025–2026

Índice

1. Introducción	3
2. Dataset empregado	3
3. GAN	3
3.1. Motivación e formulación	3
3.2. Arquitectura implementada	4
3.3. Resultados da clasificación binaria co discriminador	5
3.4. Clasificación multiclase co discriminador	8
3.5. GAN como xerador de datos sintéticos	11
4. AdaBoost	11
4.1. Clasificación binaria	12
4.2. Clasificación multiclase	15
4.3. Dataset aumentado	18
5. Comparativa	19
6. Conclusións	20
7. Traballo Futuro	21

1. Introducción

Neste traballo empréganse dous modelos de intelixencia artificial sobre o dataset CIC-IDS2017 co obxectivo de estudar a súa aplicación nun sistema de detección de intrusións a nivel de rede. Os modelos escollidos foron AdaBoost [1] como modelo base e GAN [2] como modelo complexo.

O obxectivo principal da práctica é resolver un problema de clasificación binaria, no que cada fluxo debe etiquetarse como **BENIGN** ou **ATTACK**. Ademais, como extensión, tamén se analiza o comportamento de ambos no escenario multiclase, empregando as etiquetas orixinais do dataset.

Nesta segunda parte da práctica non se realiza de novo o preprocesado completo, senón que se parte do conxunto de datos xa preparado na entrega anterior. O traballo céntrase, por tanto, no adestramento, avaliación e comparación dos modelos, así como na análise das dificultades atopadas e das prestacións acadadas en cada caso.

2. Dataset empregado

O dataset empregado neste traballo procede do preprocesado realizado na primeira práctica. Partiuse da versión completa **TrafficLabelling** de CIC-IDS2017, sobre a que se eliminaron filas baleiras, valores nulos, valores infinitos, columnas duplicadas ou sen información útil e variables con alto risco de introducir sesgo, como identificadores de fluxo, IPs e marcas temporais.

Tras a limpeza inicial, o conxunto útil quedou en 2 827 876 fluxos. As variables categóricas principais, como portos e protocolo, foron codificadas numericamente, agrupando portos pouco frecuentes nunha categoría común. As variables numéricas foron normalizadas e posteriormente realizouse unha selección de características apoiada en PCA, mantendo finalmente 52 características predictoras.

Debido ao forte desbalanceo entre clases, aplicouse unha combinación de *undersampling* nas clases maioritarias e SMOTE nas clases minoritarias. O dataset final usado nesta práctica quedou composto por 182 000 mostras. A partición realizouse de forma estratificada cun reparto 70/15/15, obtendo 127 400 mostras de adestramento, 27 300 de validación e 27 300 de test.

No escenario binario, a etiqueta **BENIGN** codificouse como clase positiva e todas as demais etiquetas como clase negativa **ATTACK**. Esta mesma partición foi empregada tanto para AdaBoost como para a GAN nos primeiros experimentos. Posteriormente, usáronse todas as etiquetas en ambos modelos.

3. GAN

3.1. Motivación e formulación

A GAN empregouse como modelo complexo da práctica. Durante o desenvolvemento probáronse varias formulacións.

Nun primeiro intento, a GAN formulouse como un detector de anomalías. Para iso, o discriminador adestrábase só con exemplos **BENIGN** do conxunto de adestramento balanceado e con mostras sintéticas xeradas a partir de ruído. A idea era que o discriminador aprendese a recoñe-

cer tráfico benigno e que, na avaliación, os fluxos cun score baixo fosen considerados anomalías ou ataques.

Despois probouse unha segunda variante do mesmo enfoque, empregando un conxunto máis amplo de exemplos malignos extraídos do dataset orixinal. Esta segunda proba foi tamén exploratoria e serviu para comparar o comportamento da GAN cando se lle fornecían só mostras benignas do conxunto balanceado fronte ao caso no que se empregaban moitos máis exemplos benignos procedentes do conxunto inicial.

A versión final implementada foi distinta. O discriminador recibe mostras reais do conxunto de adestramento e optimízase coa súa etiqueta binaria: **BENIGN** como clase positiva e **ATTACK** como clase negativa. Ademais, tamén recibe mostras sintéticas xeradas pola GAN, que se empregan como exemplos artificiais coa etiqueta 0 durante o adestramento do discriminador. Pola súa banda, o xerador optimízase para producir mostras que o discriminador clasifique como 1.

Polo tanto, a versión final non é unha GAN non supervisada clásica nin unha GAN condicionada. É unha formulación adversaria híbrida: o discriminador aproveita as etiquetas binarias reais como obxectivo da perda, pero a etiqueta non se introduce como entrada da rede.

3.2. Arquitectura implementada

A arquitectura final estrutúrase arredor de dous procesos de adestramento alternos: o adestramento do discriminador e o adestramento do xerador. O xerador recibe un vector aleatorio do espazo latente e transforma ese vector nunha mostra sintética x' . Pola súa banda, o discriminador recibe tanto mostras reais do dataset como mostras sintéticas xeradas pola GAN.

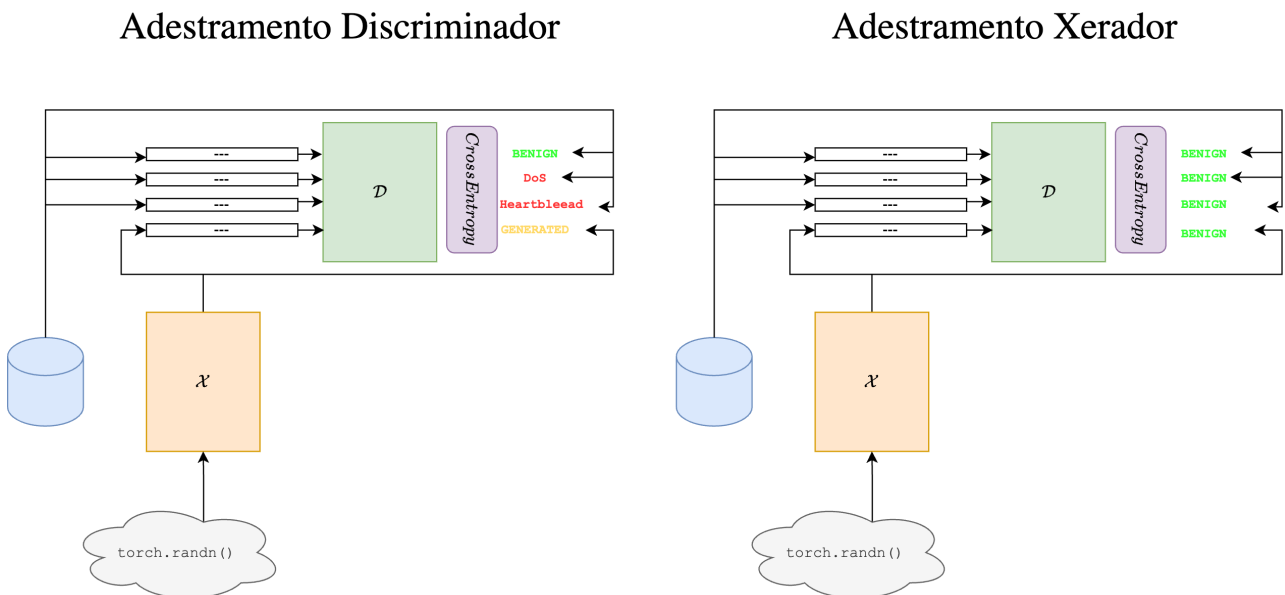


Figura 1: Esquema do adestramento adversario da GAN. Á esquerda represéntase o adestramento do discriminador, que aprende a distinguir a natureza dos exemplos (e.g. benignos, DoS, xerado, etc.). Á dereita represéntase o adestramento do xerador, no que se actualiza o xerador para producir mostras sintéticas capaces de enganar ao discriminador, xa que ó CrossEntropy dámoslle sempre a etiqueta benigno

No adestramento do discriminador, a rede recibe exemplos reais coas súas etiquetas corres-

pondentes e tamén exemplos xerados polo xerador, tratados como mostras artificiais durante o proceso de aprendizaxe. A perda emprégase para actualizar o discriminador e mellorar a súa capacidade de asignar unha saída coherente a cada tipo de mostra. No adestramento do xerador, en cambio, o discriminador actúa como avaliador: o xerador modifica os seus pesos para crear mostras x' que sexan clasificadas polo discriminador como pertencentes á clase obxectivo. Deste xeito prodúcese a dinámica adversaria propia dunha GAN.

Tamén se axustaron diferentes parámetros do adestramento, como a regularización, o tamaño dos lotes e a dimensión do espazo latente. Isto permitiu facer probas curtas para verificar o funcionamento da tubaxe e probas longas para intentar estabilizar o adestramento.

3.3. Resultados da clasificación binaria co discriminador

Realizouse un adestramento con métricas rexistradas ata a época 440. Nesta execución acadouse unha AUC-ROC máxima de validación de 0,9499 na época 310 e unha *accuracy* máxima de validación de 0,9300 na época 388. Isto mostra que a arquitectura era capaz de aprender unha fronteira útil, pero cun custo de adestramento moi superior ao de AdaBoost, como vereos na Sección 4.

Ademais das métricas de validación, tamén se avaliou o mellor discriminador da GAN sobre o conxunto de test. Nesta avaliación obtívose unha *accuracy* de 0,9327, unha *precision* de 0,9273, un *recall* de 0,9844, un F1-score de 0,9550 e unha AUC-ROC de 0,9544.

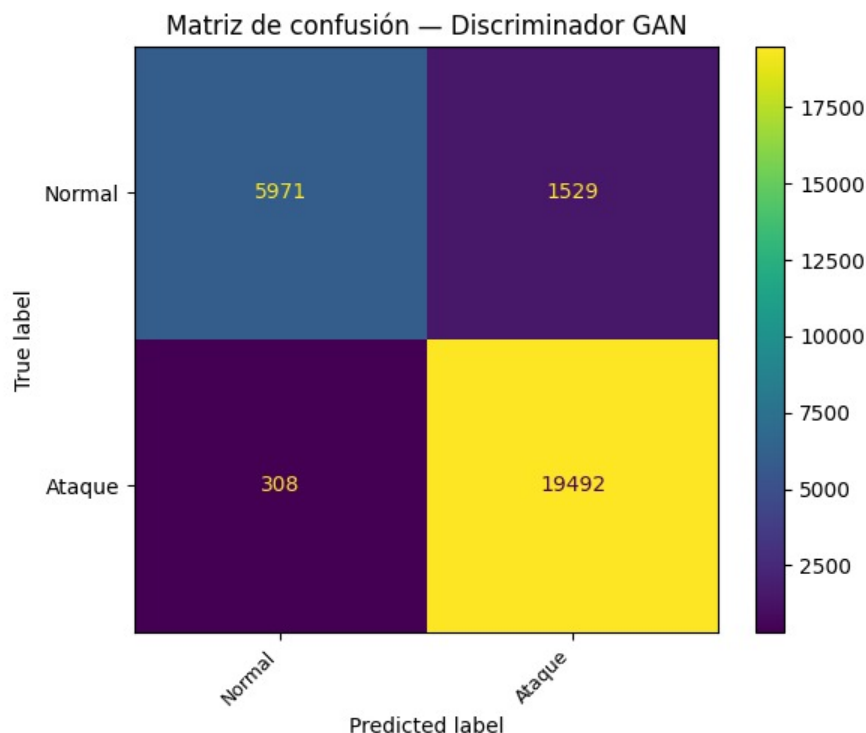


Figura 2: Matriz de confusión do discriminador da GAN sobre o conxunto de test. O modelo detecta correctamente 19 492 ataques e 5 971 fluxos normais, con 1 529 falsos positivos e 308 falsos negativos.

Como se pode ver na figura 2, a matriz de confusión mostra que a GAN ten unha capacidade moi alta para detectar ataques, xa que só deixa sen detectar 308 mostras da clase ATTACK. O

principal erro aparece nos falsos positivos: 1 529 fluxos normais son clasificados como ataques. Nun contexto de detección de intrusionés, este comportamento é razoábel, xa que é preferíbel xerar algunhas alertas adicionais antes que deixar pasar tráfico malicioso.

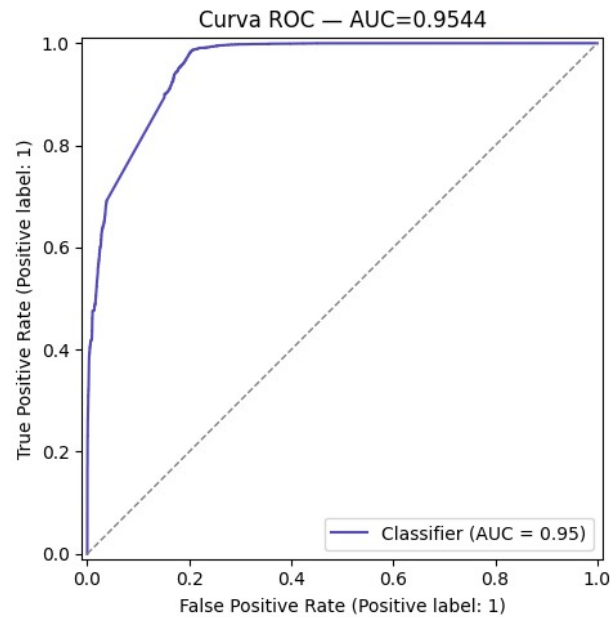


Figura 3: Curva ROC do discriminador da GAN sobre o conxunto de test. A AUC-ROC acadada é 0,9544.

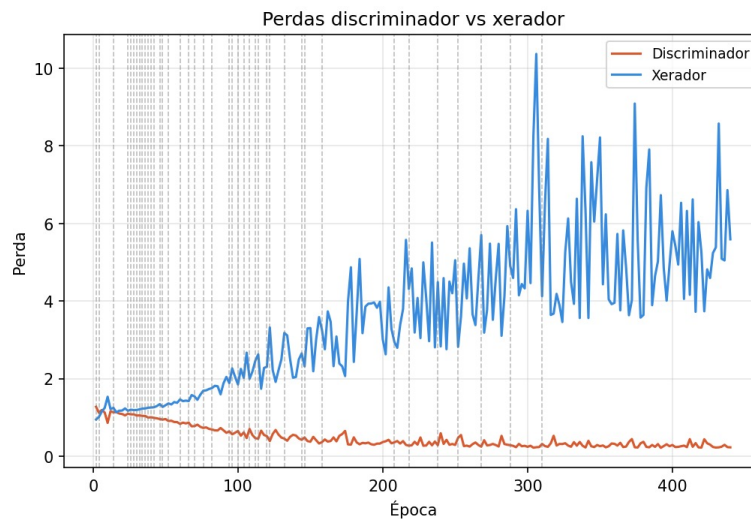


Figura 4: Evolución das perdas do discriminador e do xerador durante o adestramento longo da GAN. Obsérvase que a perda do discriminador diminúe progresivamente mentres que a do xerador tende a aumentar e presenta oscilacións considerábeis, algo habitual no adestramento adversario. As liñas verticais marcan épocas nas que se rexistraron melloras do mellor valor de validación.

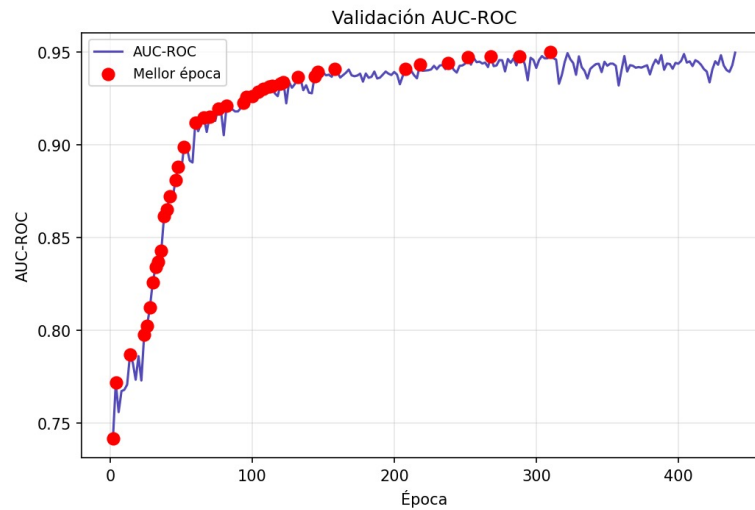


Figura 5: Evolución da AUC-ROC de validación ao longo do adestramento da GAN. Os puntos vermellos marcan as épocas nas que se mellorou o mellor valor acadado ata ese momento. Apréciase unha mellora rápida nas primeiras épocas e unha estabilización posterior arredor de 0,94–0,95, cun máximo de 0,9499.

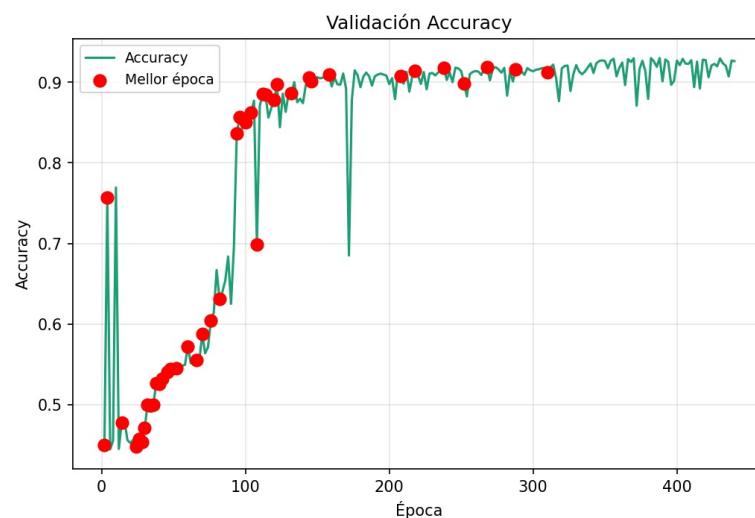


Figura 6: Evolución da *accuracy* de validación ao longo do adestramento da GAN. Os puntos vermellos indican as épocas nas que se acadou unha mellora do mellor valor ata o momento. Tras unha fase inicial inestábel, a *accuracy* estabilízase arredor de 0,92–0,93, cun máximo de 0,9300.

As tres gráficas de adestramento anteriores amosan o comportamento típico dun adestramento adversario. Nas primeiras épocas obsérvase unha mellora rápida da AUC-ROC e da *accuracy* de validación, mentres que posteriormente as curvas se estabilizan arredor de valores altos. Ao mesmo tempo, as perdas do xerador e do discriminador presentan oscilacións, o que reflicte a dificultade de equilibrar ambas redes durante o adestramento.

3.4. Clasificación multiclase co discriminador

Ademais da avaliación binaria, tamén se avaliou o discriminador da GAN no escenario multiclase, mantendo as etiquetas orixinais do dataset. Neste caso, o modelo debe distinguir non só entre tráfico benigno e ataque, senón tamén entre os distintos tipos de ataque.

Sobre o conxunto de test, o discriminador acadou unha *accuracy* de 0,6976, unha *precision macro* de 0,8912, un *recall macro* de 0,6814, un F1 macro de 0,7032 e un F1 weighted de 0,7409. Tamén se calculou a AUC-ROC por clase mediante unha estratexia one-vs-rest. A media das AUC-ROC por clase foi aproximadamente 0,966, o que indica unha boa capacidade de separación global, aínda que isto non elimina as confusións observadas posteriormente na matriz de confusión.

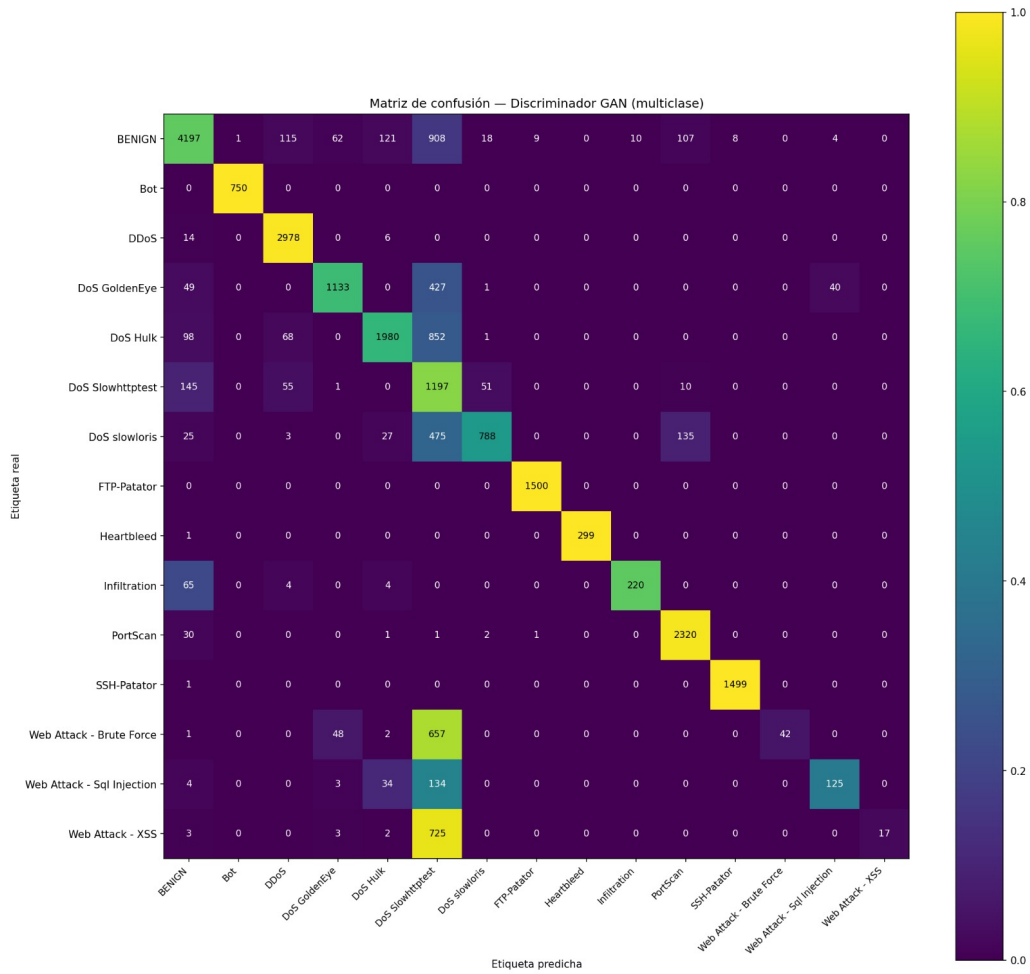


Figura 7: Matriz de confusión do discriminador da GAN no escenario multiclase. O modelo acada bos resultados en clases como Bot, DDoS, FTP-Patator, Heartbleed e SSH-Patator, pero presenta confusións importantes en clases como BENIGN, DoS Slowhttptest e varios tipos de Web Attack. A entrada (i, j) indica a $p(\text{Clase}_i | \text{Clase}_j)$.

Como se pode apreciar na figura 7, a matriz de confusión mostra que o discriminador da GAN identifica moi ben varias clases con patróns claros, especialmente Bot, DDoS, FTP-Patator, Heartbleed e SSH-Patator. Porén, tamén se observa unha forte tendencia a predicir algunhas mostras como DoS Slowhttptest, o que provoca moitos falsos positivos nesa clase. Ademais, as clases Web Attack - Brute Force e Web Attack - XSS teñen un *recall* moi baixo, polo que o modelo apenas consegue recuperalas correctamente.

A continuación recóllense as curvas ROC one-vs-rest calculadas sobre o conxunto de test, así como a evolución das métricas de validación durante o adestramento multiclase.

Táboa 1: AUC-ROC one-vs-rest do discriminador da GAN no escenario multiclase

Clase	AUC-ROC
BENIGN	0,9412
Bot	1,0000
DDoS	0,9983
DoS GoldenEye	0,9661
DoS Hulk	0,9603
DoS Slowhttptest	0,9153
DoS slowloris	0,9281
FTP-Patator	1,0000
Heartbleed	1,0000
Infiltration	0,9898
PortScan	0,9926
SSH-Patator	0,9999
Web Attack - Brute Force	0,9256
Web Attack - Sql Injection	0,9444
Web Attack - XSS	0,9228

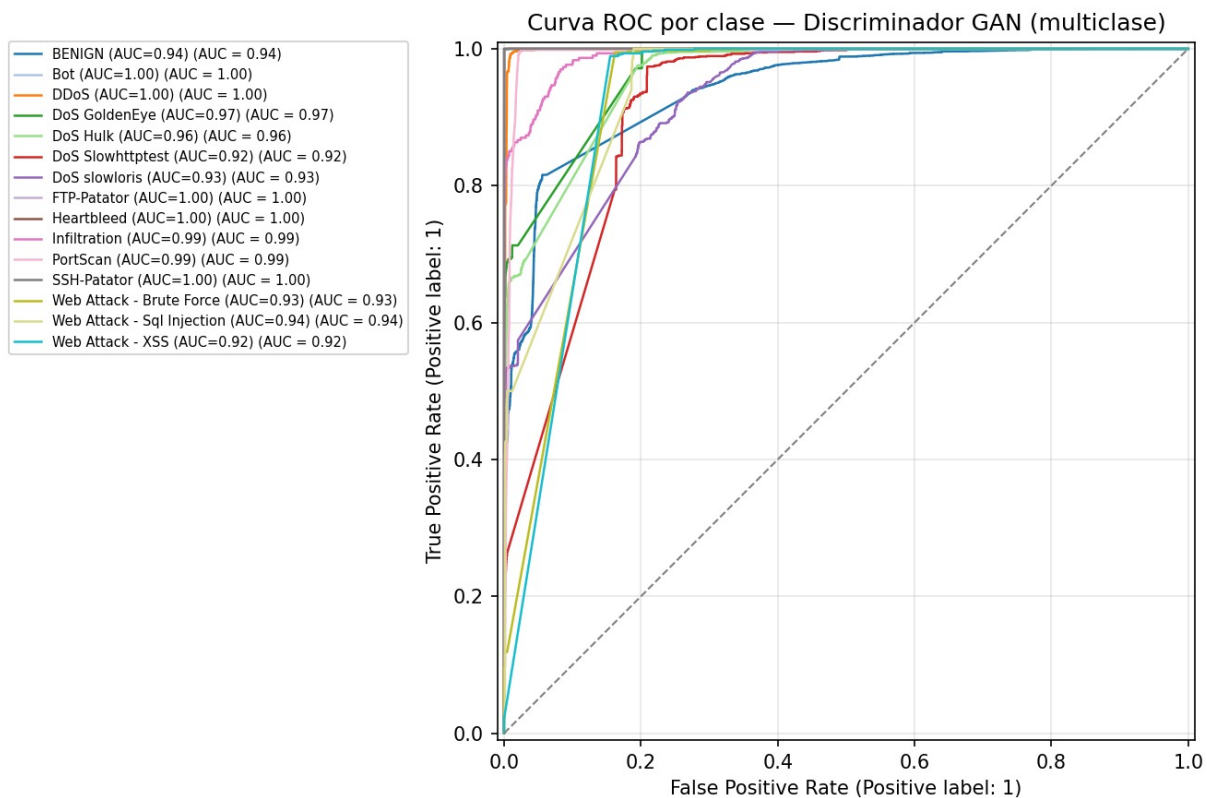


Figura 8: Curvas ROC one-vs-rest do discriminador da GAN no escenario multiclase. A maioría das clases presentan unha AUC-ROC elevada, con valores próximos ou superiores a 0,92 en todas as clases.

As curvas ROC mostran que o discriminador da GAN separa ben cada clase fronte ao resto cando se analiza o problema en formato one-vs-rest. Destacan clases como **Bot**, **FTP-Patator**, **Heartbleed** e **SSH-Patator**, con valores practicamente perfectos. Porén, estes bos valores de AUC-ROC deben interpretarse xunto coa matriz de confusión, xa que unha boa separación por puntuación non implica necesariamente que todas as mostras rematen clasificadas correctamente na clase final.

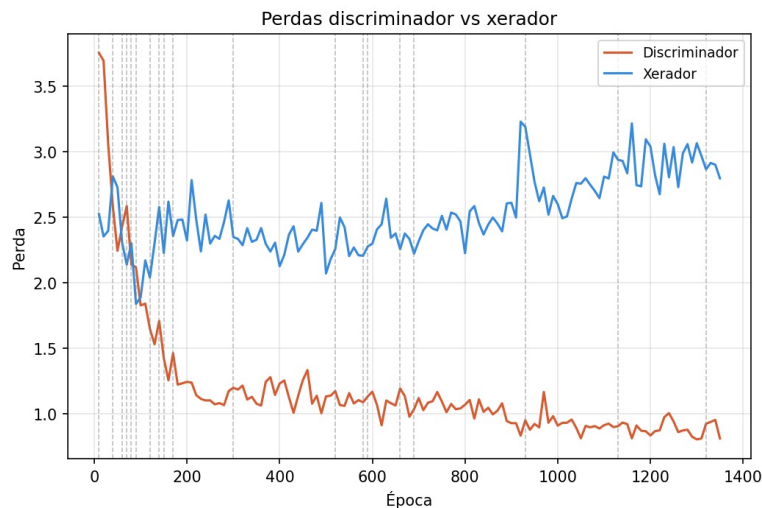
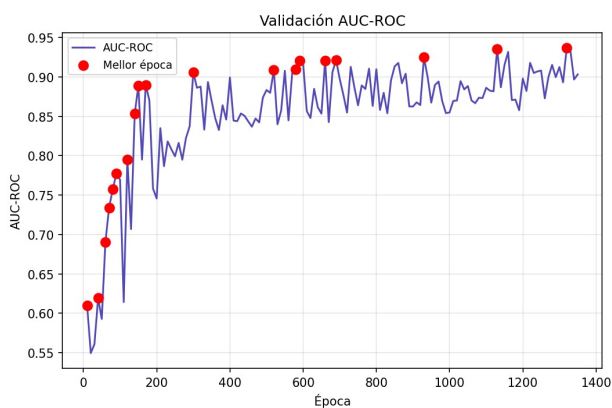
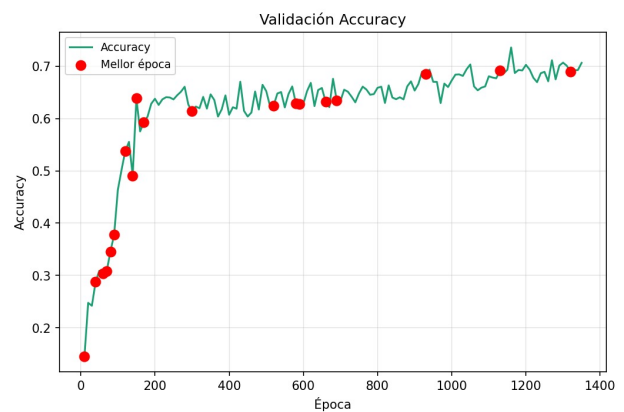


Figura 9: Evolución das perdas do discriminador e do xerador durante o adestramento multi-clase da GAN. A perda do discriminador diminúe de forma clara nas primeiras épocas e despois estabilízase, mentres que a perda do xerador presenta oscilacións e tende a aumentar, reflectindo a dificultade de manter equilibrado o adestramento adversario. Sábese por teoría [2] que eventualmente, dada a natureza de xogo de ganancia cero, o xerador comezaría a mellorar a perda. Non obstante, aquí non se observa isto, xa que os experimentos non foron suficientemente longos por limitación de recursos.



(a) Evolución da AUC-ROC de validación durante o adestramento multi-clase da GAN. Obsérvase unha mellora rápida nas primeiras épocas e unha estabilización posterior arredor de valores altos, con oscilacións propias do adestramento adversario.



(b) Evolución da *accuracy* de validación durante o adestramento multi-clase da GAN. A métrica mellora progresivamente desde as primeiras épocas e estabilízase arredor de 0,70, valor coherente coa accuracy obtida posteriormente sobre o conxunto de test.

Figura 10: Métricas de validación durante o adestramento multi-clase da GAN.

Estas curvas mostran que o adestramento multi-clase tamén precisa moitas épocas para estabili-

zarse. A AUC-ROC de validación acadou valores altos, pero a *accuracy* queda arredor de 0,70, o que indica que o modelo aprende a separar razoabelmente as clases, aínda que segue cometendo confusións entre ataques con patróns próximos ou con menor representación.

En comparación co escenario binario, o problema multiclase resulta máis complexo, xa que pequenas diferenzas entre ataques similares poden xerar confusións entre clases. Aínda así, os resultados multiclase da GAN son superiores aos obtidos por AdaBoost en F1 macro e F1 weighted, o que indica que o discriminador consegue unha separación máis equilibrada entre clases.

Ademáis, como observamos que as gráficas de perda continúan baixando, e as de avaliación subindo, sabemos que con máis épocas acadaríamos mellores resultados. Porén, polo alcance da práctica e dos recursos dispoñibles, decidíuse non continuar co adestramento.

3.5. GAN como xerador de datos sintéticos

Un dos engadidos máis interesantes do traballo foi a implementación dunha tubaxe de *data augmentation*. Para iso, empregouse o mellor xerador da GAN para crear novas mostras sintéticas e incorporalas ao conxunto de adestramento, mantendo sen cambios os conxuntos de validación e test.

O `script` que nos permitiu xerar o conxunto de datos tiña un parámetro de proporción que permitía controlar cantas mostras reais se mantiñan no novo conxunto de adestramento e cantas mostras sintéticas se engadían. Nunha das execucións xeráronse 127 400 mostras sintéticas adicionais, polo que o novo conxunto de adestramento pasou de 127 400 a 254 800 mostras.

As mostras xeradas identificáronse como sintéticas no novo conxunto de datos. Posteriormente, este dataset aumentado empregouse para volver avaliar AdaBoost no escenario binario, mantendo os mesmos conxuntos de validación e test. Deste xeito puido comprobarse se a augmentación con mostras xeradas pola GAN melloraba ou non o rendemento do modelo base.

4. AdaBoost

AdaBoost foi o modelo base escollido para a práctica. Trátase dun modelo supervisado sinxelo, rápido de adestrar e axeitado para datos tabulares, polo que serve como referencia para comparar fronte á GAN. No que segue, úsase como estimador base do AdaBoost unha árbore de decisións.

Realizouse unha primeira procura en grella, tanto no caso binario como no multiclase. Esta proba permitiu comprobar que AdaBoost funcionaba correctamente sobre o dataset preprocesado e obter unha primeira estimación do rango de hiperparámetros útil.

Na versión final empregouse optimización bayesiana para automatizar a procura de hiperparámetros, onde algúns deles non admitían valores flotantes [3, 4]. O espazo de busca incluíu o número de estimadores, con valores entre 15 e 500, e a taxa de aprendizaxe, con valores entre 0.0001 e 2. A execución final realizouse con 15 puntos aleatorios iniciais que supuxeron unha búsqueda aleatoria no espazo acotado, e 40 iteracións de optimización realizadas polo algoritmo. A función obxectivo foi o F1 micro en validación. Esta elección débese a que ten en conta o desbalanceo de clases.

AdaBoost avaliou-se en dous escenarios diferentes. No primeiro empregáronse as etiquetas binarias, distinguindo entre **BENIGN** e **ATTACK**. No segundo mantivéronse as etiquetas multiclase orixinais do dataset. Deste xeito puido estudarse o comportamento do mesmo modelo tanto nun problema binario como nun problema de clasificación máis detallado.

4.1. Clasificación binaria

No escenario binario, AdaBoost obtivo un resultado sólido sobre o conxunto de test, cunha *accuracy* de 0,9144 e un F1 micro de 0,9144. O modelo amosou un *recall* moi alto para a clase **ATTACK**, de 0,98, sacrificando parte do *recall* da clase **BENIGN**, que quedou en 0,73. Nun contexto de ciberseguridade, esta compensación é razoábel, xa que adoita ser preferíbel asumir máis falsos positivos antes que deixar pasar tráfico malicioso.

O reparto do problema binario no dataset final foi o seguinte: 35 000 mostras **BENIGN** e 92 400 **ATTACK** no adestramento, e 7 500 / 19 800 tanto en validación como en test.

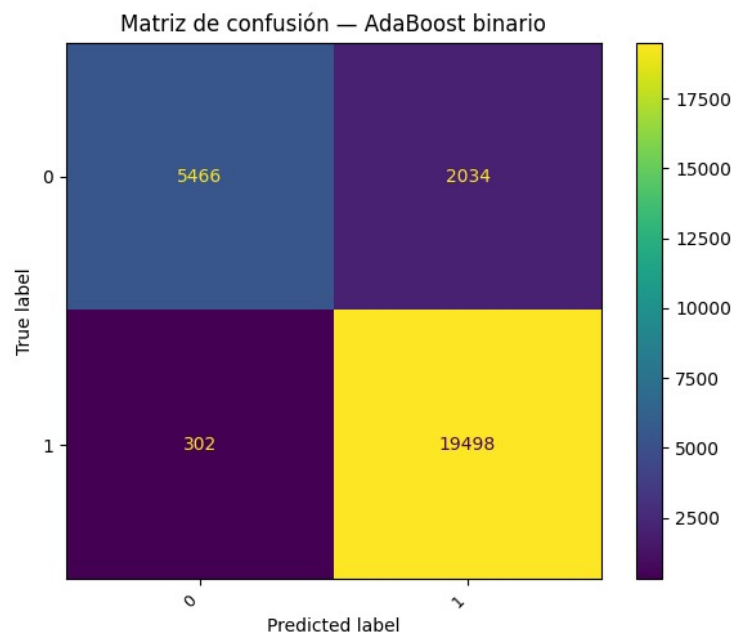


Figura 11: Matriz de confusión de AdaBoost no escenario binario. O modelo clasifica correctamente 5 466 mostras benignas e 19 498 ataques, con 2 034 falsos positivos e 302 falsos negativos.

Obsérvase na Figura 11 que a matriz de confusión de AdaBoost amosa un comportamento semellante ao da GAN: o modelo detecta case todos os ataques, con só 302 falsos negativos. Porén, confunde unha parte maior do tráfico benigno, xerando 2 034 falsos positivos. Isto explica que o *recall* da clase **ATTACK** sexa moi alto, mentres que o *recall* da clase **BENIGN** queda máis penalizado.

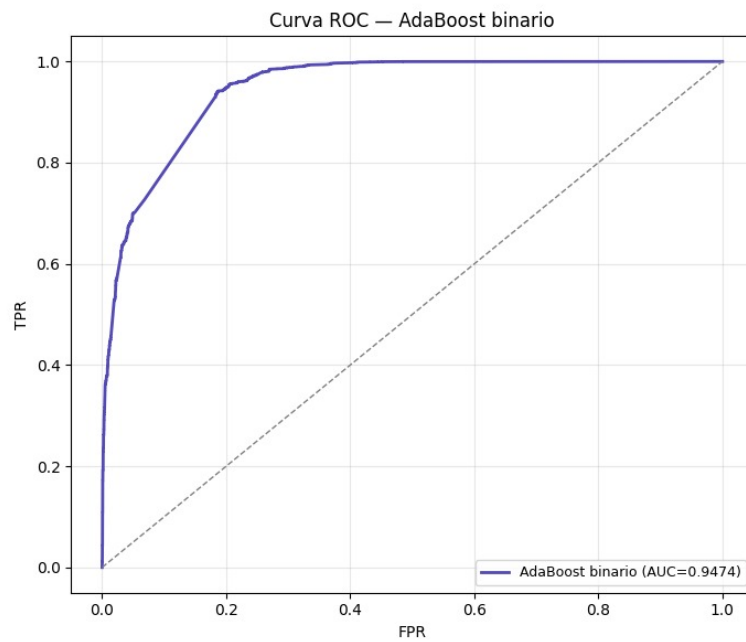


Figura 12: Curva ROC de AdaBoost no escenario binario. A AUC-ROC acadada sobre o conxunto de test é 0,9474.

As figuras seguintes recollen a evolución da busca bayesiana no caso binario. Obsérvase a converxencia do máximo acumulado e a localización das mellores combinacións no espazo de parámetros.

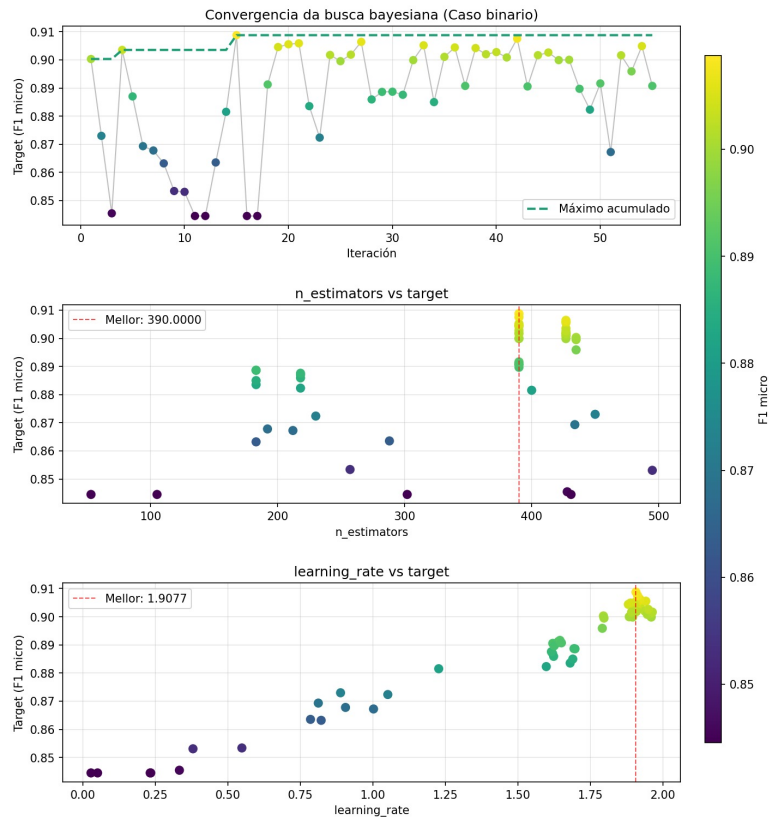


Figura 13: (Enriba) Convergencia da busca bayesiana de AdaBoost no caso binario. O mellor valor aproxímase a F1 micro $\approx 0,909$, con `n_estimators` próximo a 390 e `learning_rate` arredor de 1,908. (Medio e Abaixo) Resposta marxinal da función obxectivo con respecto a cada hiperparámetro.

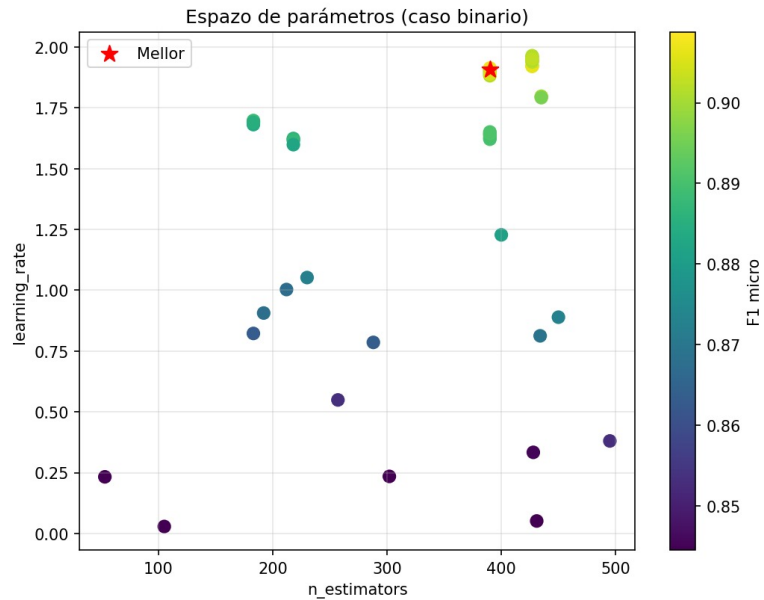


Figura 14: Espazo de parámetros explorado para AdaBoost no caso binario. A concentración de puntos de maior calidade aparece na zona de `learning_rate` alto e número de estimadores medio-alto.

4.2. Clasificación multiclase

Tamén se estudou unha extensión multiclase. Neste escenario mantéñense as etiquetas orixinais do dataset, polo que o problema é máis difícil ca no caso binario: ademais de distinguir tráfico benigno fronte a ataque, o modelo debe identificar o tipo concreto de tráfico ou ataque.

Na avaliación multiclase, AdaBoost acadou unha *accuracy* de 0,65, cun *macro F1* de 0,47 e un *weighted F1* de 0,62. As clases maioritarias e algunhas firmas específicas como SSH-Patator, PortScan ou DDoS tiveron un comportamento aceptábel, pero as clases moi raras ou máis difíciles, como Heartbleed, Web Attack - Brute Force, Web Attack - Sql Injection e Web Attack - XSS, seguiron moi penalizadas.

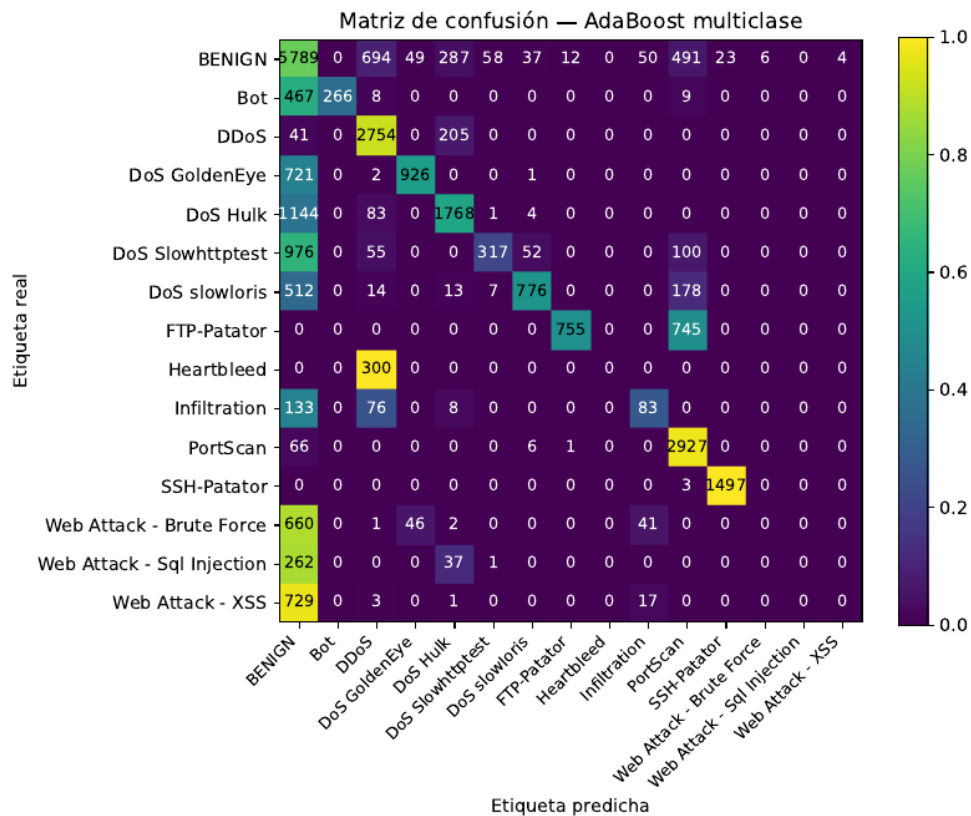


Figura 15: Matriz de confusión de AdaBoost no escenario multiclase. Obsérvase un bo comportamento en clases como DDoS, PortScan e SSH-Patator, mentres que varias clases minoritarias quedan confundidas con BENIGN ou con outras clases maioritarias.

No caso multiclase obsérvase na Figura 15 unha dificultade maior. O modelo identifica ben algunhas clases con patróns máis definidos, como SSH-Patator, DDoS ou PortScan, pero varias clases minoritarias aparecen mal clasificadas ou directamente absorbidas por clases máis frecuentes. Isto é coherente coas métricas macro, que son máis baixas ca as ponderadas, e indica que o rendemento global está influído polas clases maioritarias.

Táboa 2: AUC-ROC one-vs-rest de AdaBoost no escenario multiclase

Clase	AUC-ROC
BENIGN	0,7956
Bot	1,0000
DDoS	0,9506
DoS GoldenEye	0,9310
DoS Hulk	0,8956
DoS Slowhttptest	0,8387
DoS slowloris	0,9199
FTP-Patator	0,9998
Heartbleed	0,9977
Infiltration	0,9205
PortScan	0,8888
SSH-Patator	0,9994
Web Attack - Brute Force	0,9073
Web Attack - Sql Injection	0,8601
Web Attack - XSS	0,9179

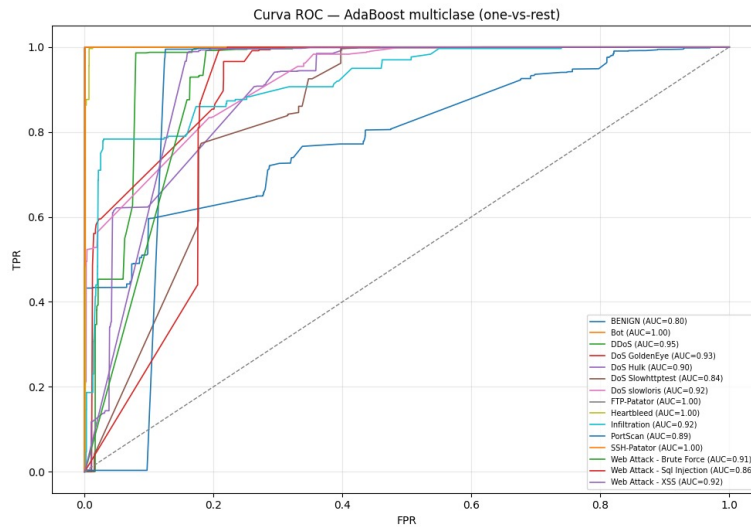


Figura 16: Curvas ROC one-vs-rest de AdaBoost no escenario multiclase. A maioría das clases presentan unha AUC elevada, aínda que a clase BENIGN obtén un valor menor, con AUC-ROC de 0,7956.

As figuras seguintes recollen a evolución da busca bayesiana no caso multiclase.

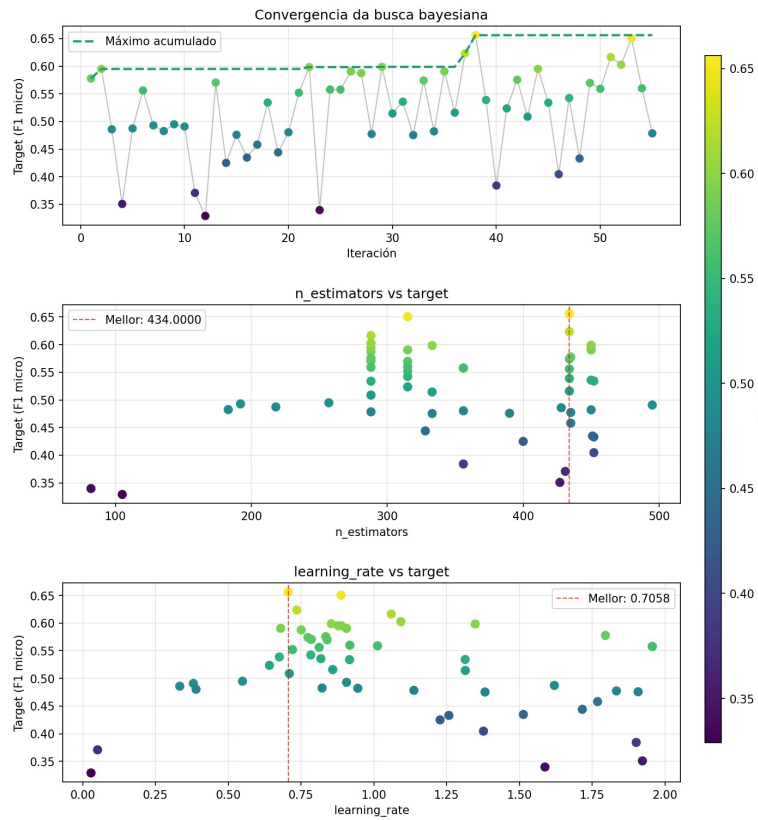


Figura 17: Convergencia da busca bayesiana de AdaBoost no caso multiclase. Obsérvase un mellor valor arredor de F1 micro $\approx 0,656$, con `n_estimators` próximo a 434 e `learning_rate` arredor de 0,706.

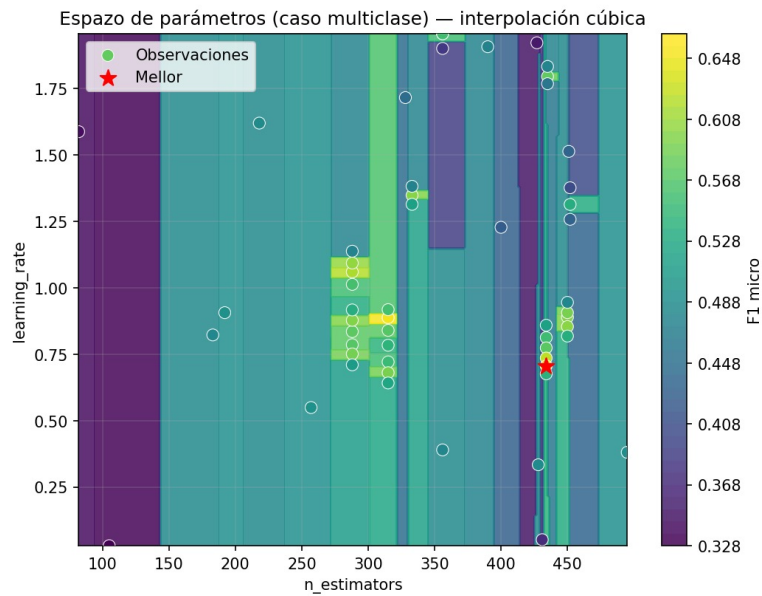


Figura 18: Espazo de parámetros explorado para AdaBoost no caso multiclase. A estrela sinala a mellor combinación atopada pola optimización bayesiana. Xa que tense os puntos do espazo de hiperparámetros e a súa correspondente avaliación da perda, pódese estimar mediante interpolación por cercanía (**nearest**) a función obxectivo que se quere maximizar. Notar a non suavidade, xa que a optimización está en parámetros enteros. Para detalles do algoritmo, ver [4]

4.3. Dataset aumentado

A continuación de empregar a GAN como xerador, avalíase tamén o uso de mostras sintéticas como técnica de *data augmentation*. O obxectivo era comprobar se engadir datos xerados pola GAN ao conxunto de adestramento podía mellorar o rendemento de AdaBoost no escenario binario.

Nunha das execucións conservadas xeráronse 127 400 mostras sintéticas adicionais, polo que o conxunto de adestramento pasou de 127 400 a 254 800 mostras. Mantivéronse os mesmos conxuntos de validación e test, de maneira que a comparación co AdaBoost adestrado co dataset orixinal fose directa.

O resultado obtido foi moi semellante ao do AdaBoost adestrado co dataset orixinal. Co dataset orixinal obtívose unha *accuracy* de 0,9144 e un F1 weighted de 0,9106, mentres que co dataset aumentado a *accuracy* foi de 0,9112 e o F1 weighted de 0,9078. Polo tanto, a augmentación con datos sintéticos non produciu unha mellora apreciable neste experimento.

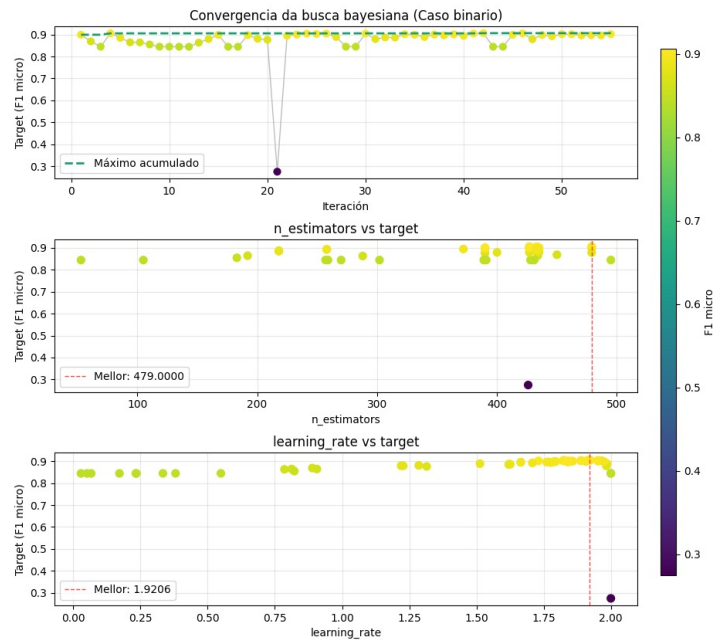


Figura 19: Convergencia da busca bayesiana de AdaBoost no caso binario empregando o dataset aumentado. O mellor valor mantense arredor de F1 micro $\approx 0,91$, moi próximo ao obtido co dataset orixinal.

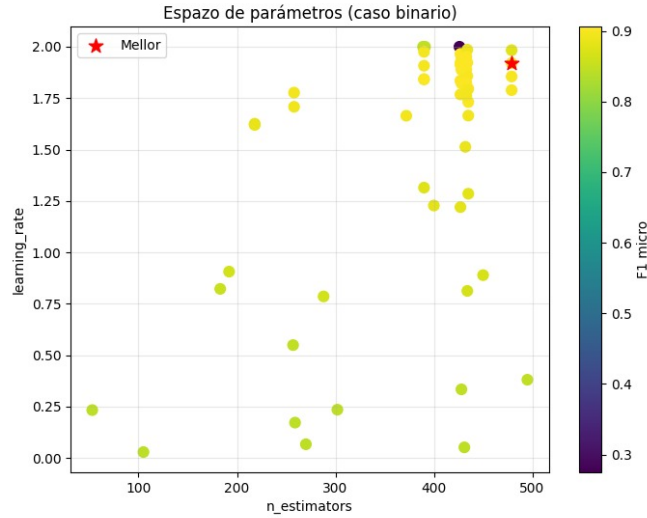


Figura 20: Espazo de parámetros explorado para AdaBoost no caso binario co dataset aumentado. A mellor configuración aparece nunha zona de número alto de estimadores e *learning rate* elevado, semellante ao observado no dataset orixinal.

A explicación máis probábel é que o xerador aínda non reproduce con suficiente fidelidade a distribución real dos datos. No adestramento dunha GAN, primeiro é habitual que o discriminador aprenda a separar mellor as mostras reais das xeradas; só despois, cando esa sinal de erro é útil, o xerador comeza a mellorar progresivamente. Aínda que o discriminador acadou bos resultados como clasificador, iso non garante que as mostras sintéticas xeradas teñan a mesma calidade estatística ca os datos reais. Por este motivo, ao engadilas ao adestramento de AdaBoost, o efecto final foi practicamente neutro.

5. Comparativa

A comparación entre ambos modelos pode facerse con métricas de test tanto no escenario binario como no escenario multiclase. AdaBoost ofrece un modelo base rápido e estábel, mentres que a GAN require máis axuste e un adestramento máis longo, pero acada mellores métricas finais no test binario conservado.

En primeiro lugar, AdaBoost foi o mellor modelo base do traballo. A súa principal vantaxe é a eficiencia: adéstrase rápido, é doado de optimizar e ofrece un rendemento moi competitivo no problema binario. Ademais, a interpretación das súas métricas é directa e permite construír unha memoria máis limpa e reproducíbel. Ademais, conta coa prestación de ser un modelo interpretable e explicable.

En segundo lugar, a GAN xustifica a súa presenza como modelo complexo precisamente porque introduce dificultades reais de adestramento. As probas curtas demostraron que a converxencia non chega en poucas épocas; o *dropout*, a configuración do adestramento e o número de iteracións inflúen moito. Por outra banda, cando se lle adica tempo dabondo (non explorado nesta práctica), o discriminador consegue métricas altas en validación, chegando a unha AUC-ROC próxima a 0,95, e o xerador abre a porta á creación de novos datasets.

Modelo	Accuracy	Precision ATTACK	Recall ATTACK	F1 ATTACK	AUC-ROC
AdaBoost	0,9144	0,91	0,98	0,94	0,9474
AdaBoost con dataset aumentado	0,9112	0,91	0,98	0,94	0,9477
GAN	0,9327	0,9273	0,9844	0,9550	0,9544

Táboa 3: Comparativa cuantitativa no escenario binario sobre test

O AdaBoost adestrado co dataset aumentado obtivo métricas moi próximas ás do AdaBoost adestrado só cos datos reais. Isto indica que, nesta configuración, as mostras sintéticas xeradas pola GAN non achegaron información adicional suficiente para mellorar a fronteira de decisión do modelo base. Isto é coherente coas gráficas de perda do adestramento da GAN nas que se observa que o xerador aínda non converxera.

Modelo	Accuracy	Precision macro	Recall macro	F1 macro	F1 weighted	AUC-ROC media
AdaBoost multiclase	0,65	0,58	0,45	0,47	0,62	0,9215
GAN multiclase	0,6976	0,8912	0,6814	0,7032	0,7409	0,9656

Táboa 4: Comparativa cuantitativa no escenario multiclase sobre test

No escenario multiclase, a GAN obtivo mellores métricas globais ca AdaBoost, especialmente en F1 macro. Isto indica que o discriminador tivo un comportamento máis equilibrado entre clases, aínda que seguiu presentando problemas en clases concretas, como algúns ataques web e certas variantes de DoS.

En resumo, se se valora a simplicidade, a velocidade e a estabilidade do adestramento, AdaBoost é a opción máis práctica. Se se atende ás métricas obtidas no test, a GAN acadou mellores métricas globais tanto no escenario binario como no multiclase, aínda que cun custo de axuste e adestramento maior. Ademais, a GAN achega unha utilidade adicional ao permitir xerar mostras sintéticas.

6. Conclusións

O traballo permitiu completar o obxectivo principal da práctica: adestrar e comparar dous modelos diferentes sobre CIC-IDS2017, partindo do preprocesado realizado na primeira entrega e analizando os resultados obtidos.

AdaBoost consolidouse como unha solución base moi forte no escenario binario. Coa configuración axeitada ofreceu métricas altas, un tempo de execución reducido e algo máis de interpretabilidade que a GAN. Por iso pode considerarse o modelo máis práctico e estábel do traballo, aínda que a GAN acadase mellores métricas no test binario conservado.

A GAN foi máis esixente. As probas iniciais non chegaron a converxer, pero os adestramentos longos demostraron que o discriminador podía acadar resultados competitivos tanto no escenario binario como no multiclase, con mellores métricas globais ca AdaBoost no test multiclase conservado. Porén, non se conseguiu a converxencia do xerador que, en parte, é un dos atractivos do modelo GAN. O feito de reutilizar o xerador para producir novas mostras permitiu avaliar a GAN como ferramenta de augmentación de datos, aínda que nesta execución concreta non se obtivo unha mellora apreciable en AdaBoost.

A conclusión global é que AdaBoost ofrece a mellor relación entre simplicidade, estabilidade e custo de adestramento, mentres que a GAN acada mellores métricas globais nos escenarios avaliados, tanto no test binario como no test multiclase conservado, e introduce flexibilidade adicional ao permitir a xeración de mostras sintéticas. Esa combinación fai que a comparativa entre ambos modelos sexa rica e xustificábel.

7. Traballo Futuro

Explorar con experimentos longos variantes máis específicas da GAN, como GANs condicionais ou unha GAN independente por clase, co obxectivo de mellorar o comportamento nas clases multiclase máis difíciles. Ademais, esa GAN condicional, unha vez adestrada, podería ser de utilidade (independentemente do seu rendemento en clasificación multiclase) como xerador de mostras sintéticas para adestrar un AdaBoost multiclase mellorado.

Referencias

- [1] Y. Freund and R. E. Schapire, “Experiments with a new boosting algorithm,” in *Machine Learning: Proceedings of the Thirteenth International Conference*. Morgan Kaufmann, 1996, pp. 148–156.
- [2] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Advances in Neural Information Processing Systems*, vol. 27, 2014.
- [3] F. Nogueira, “Bayesian Optimization: Open source constrained global optimization tool for Python,” 2014–. [Online]. Available: <https://github.com/bayesian-optimization/BayesianOptimization>
- [4] E. C. Garrido-Merchán and D. Hernández-Lobato, “Dealing with categorical and integer-valued variables in bayesian optimization with gaussian processes,” *Neurocomputing*, vol. 380, pp. 20–35, Mar. 2020. [Online]. Available: <http://dx.doi.org/10.1016/j.neucom.2019.11.004>